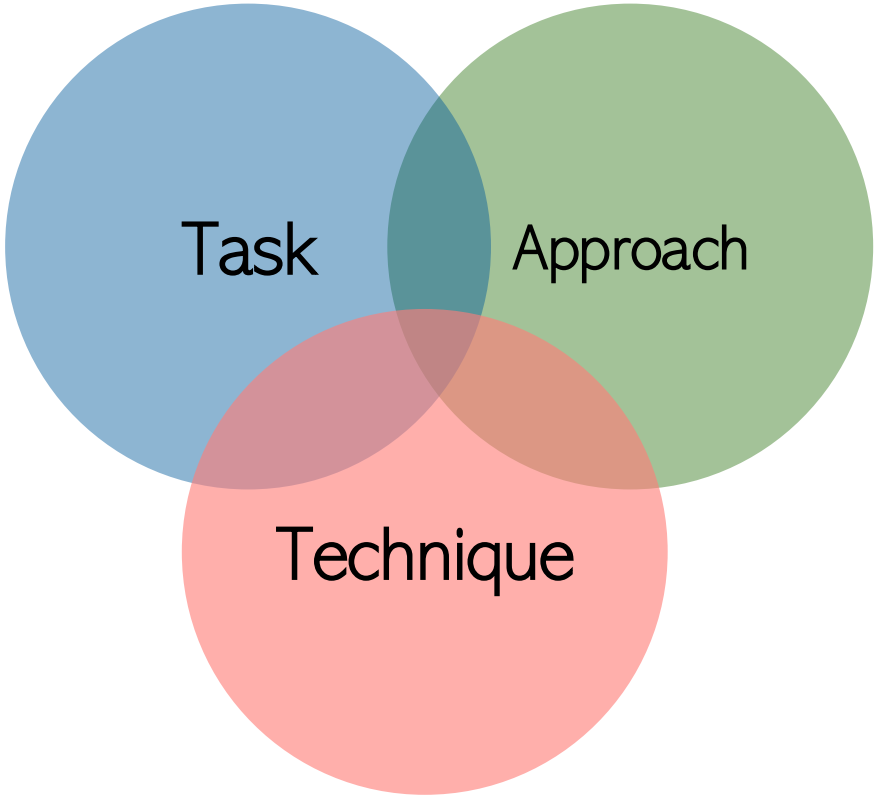
1 Outline: W07C: New Technique: Logistic Regression (LR)

Classification
Regression
Clustering
Ranking (Recommendation)
Vector Representation and Dimensionality Reduction
Language Modeling



Full/Self/Semi-**Supervised Learning**
Unsupervised Learning
Reinforcement Learning

k-Nearest Neighbor, Naive Bayes, **Logistic Regression**
Artificial Neural Networks, Graph Neural Network
Support Vector Machines
Principal Component Analysis
K-Means

Outline: W08A

I) Task: Classification

II) Supervision: Fully Supervised

III) Technique: Logistic Regression

Learning the Discriminator from Training Data (from Experience)

Mountain Descending

IIII) Evaluation

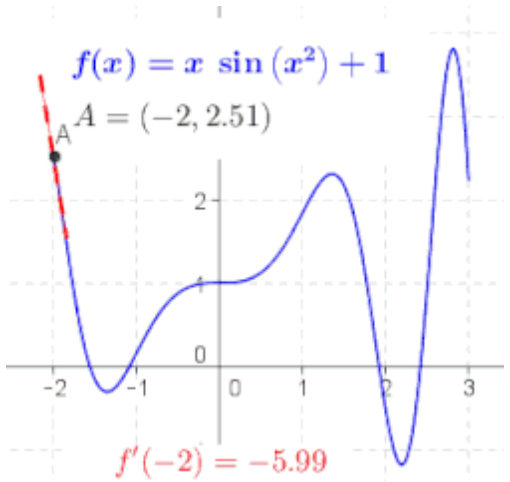
Metrics (Few New Metrics), Methodology

An Optimization Problem

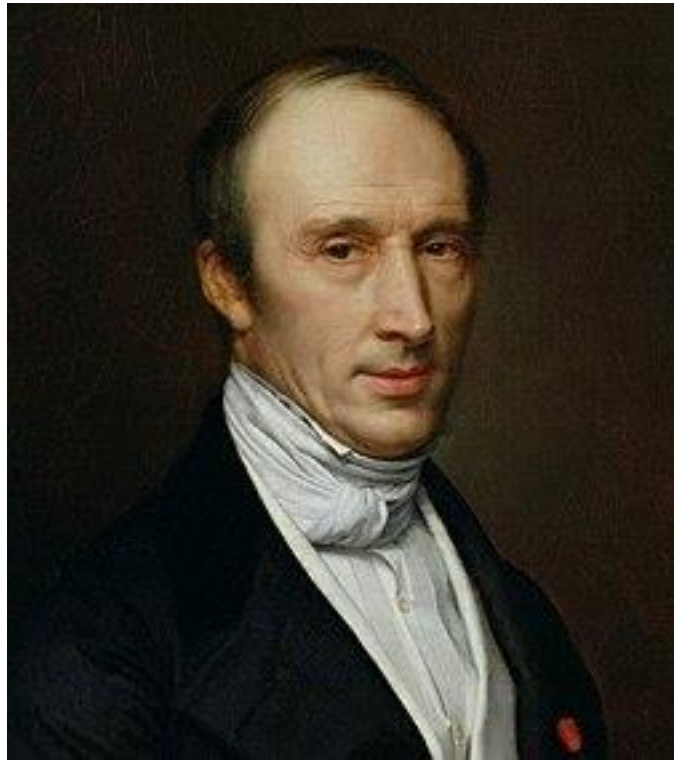


Pierre de Fermat (1700 – 1665)
French magistrate, polymath, and above all, mathematician also a poet
He is notably credited with Fermat's principle in optics.
He is especially known for having stated Fermat's Last Theorem, the proof was only established more than 300 years later.

Fermat noticed that at the very peak (maximum) or very trough /traaf/ (minimum) of a curve, the curve briefly flattens out. If you shift your position by an infinitely small amount, the output value barely changes



Baron Augustin-Louis Cauchy (1789 – 1857)
French mathematician, engineer, and physicist.
Proved the key theorems of calculus more than any other mathematician.
Approximately **800 research articles and 5 complete textbooks**.



While Fermat mastered finding minimums for single-variable equations, optimizing complex functions with multiple variables required the gradient. Cauchy invented the **Gradient Descent algorithm** to solve complex astronomical and algebraic equations, recognizing that moving in the **opposite direction of the gradient vector** provides the **steepest route** to a minimum point.

4

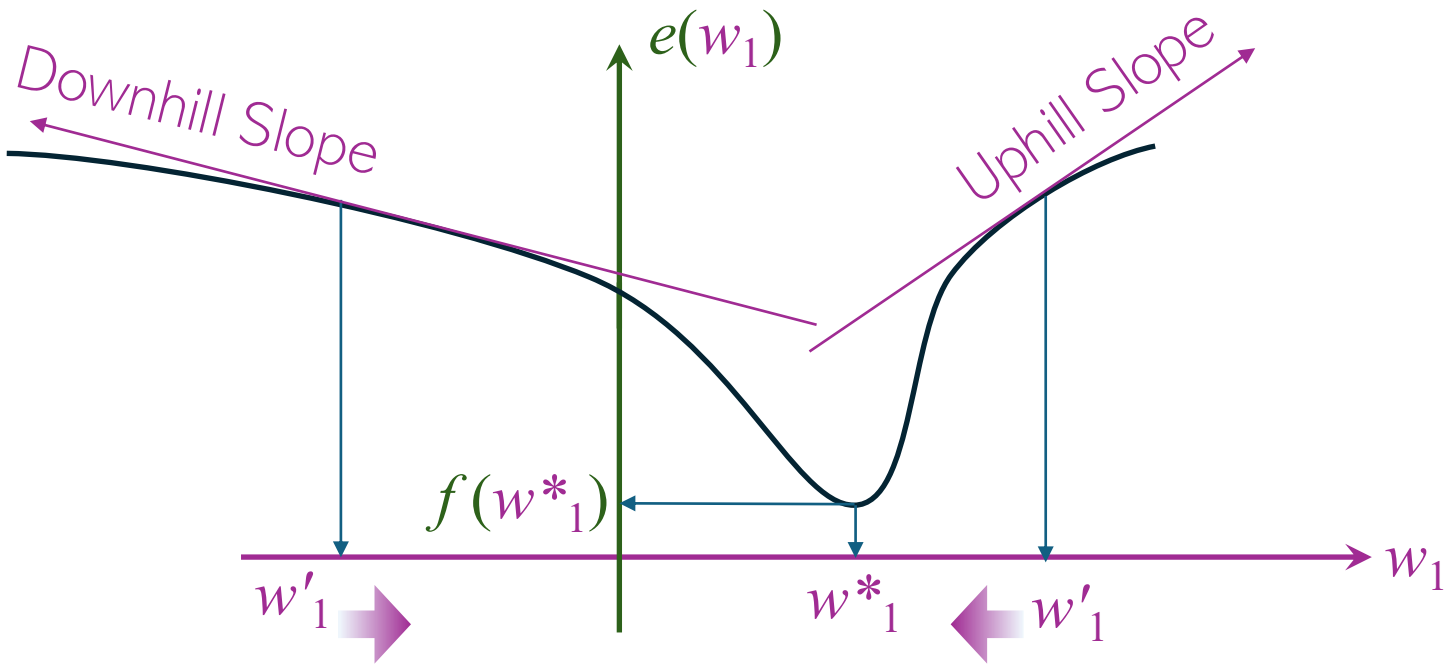
Back to Data Mining Task: Minimizing LR Error

Differentiation, Derivative

Where $e(w_1)$ become minimum value?

At what $w_1 = w^*_1$, $e(w_1 = w^*_1)$ become minimum value?

If at w'_1, how to modify $w'_1 \rightarrow w^*_1$ such that $e(w_1)$ get closer to minimum?



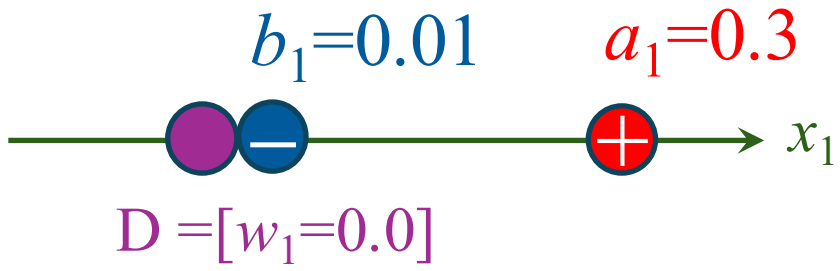
One step down:

$$w'_1 \rightarrow w'_1 - \frac{d\ e(w_1)}{d\ w_1}$$

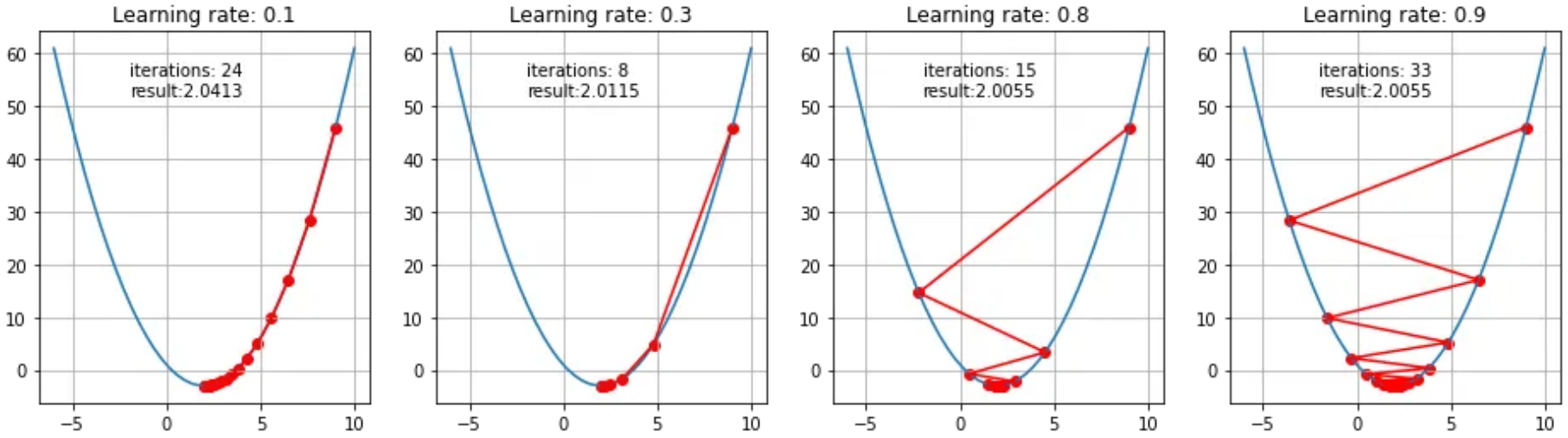
5

Learning the Discriminator: Dynamic Steps: Learning Rate

$$e(w_1) = [1.0 - f^+(\boldsymbol{o}^+)] + [f^+(\boldsymbol{o}^-) - 0.0]$$
$$= [1.0 - f^+(a_1)] + [f^+(b_1)]$$
$$= [1.0 - \sigma(0.3 - w_1)] + [\sigma(0.01 - w_1)]$$



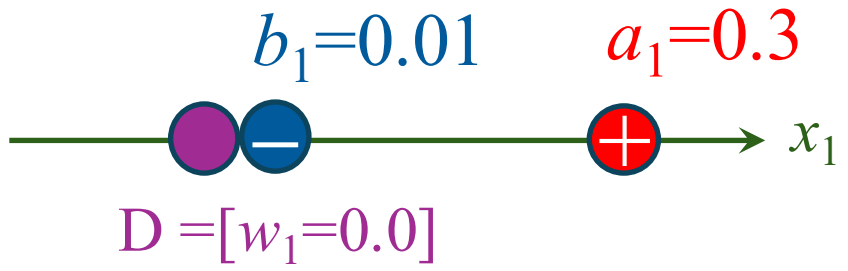
$$w^{(i+1)}{}_1 \rightarrow w^{(i)}{}_1 - \eta \times \frac{d\, e(w^{(i)}{}_1 : (a_1, b_1))}{d\, w_1}$$



https://towardsdatascience.com/gradient-descent-algorithm-a-deep-dive-cf04e8115f21

6 Learning the Discriminator: Early Stopping w/ Patience

$$e(w_1) = [1.0 - f^+(\boldsymbol{o}^+)] + [f^+(\boldsymbol{o}^-) - 0.0]$$
$$= [1.0 - f^+(a_1)] + [f^+(b_1)]$$
$$= [1.0 - \sigma(0.3 - w_1)] + [\sigma(0.01 - w_1)]$$



After certain number of steps (patience: e.g., 3),

→ No (substantial) drop to error or increasing error → Underfitting

→ Dropping errors but then plateau and then increasing

→ Stop Training

→ $w^{(0)}{}_1$ → $w^{(1)}{}_1$ → $w^{(2)}{}_1$ → $w^{(3)}{}_1$ → $w^{(4)}{}_1$

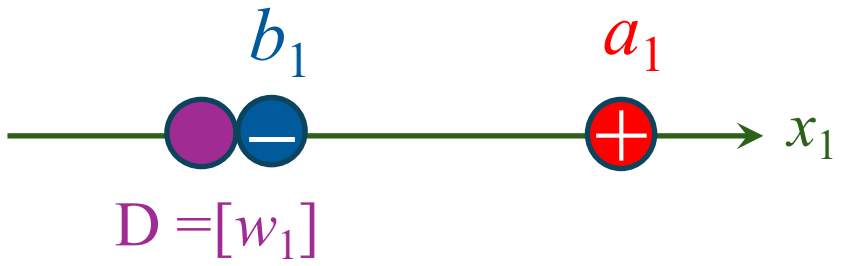
→ $e(w^{(0)}{}_1)$=0.99 → $e(w^{(1)}{}_1)$=0.85 → $e(w^{(2)}{}_1)$=0.73 → $e(w^{(3)}{}_1)$=0.72 → $e(w^{(4)}{}_1)$=0.75 → Stop

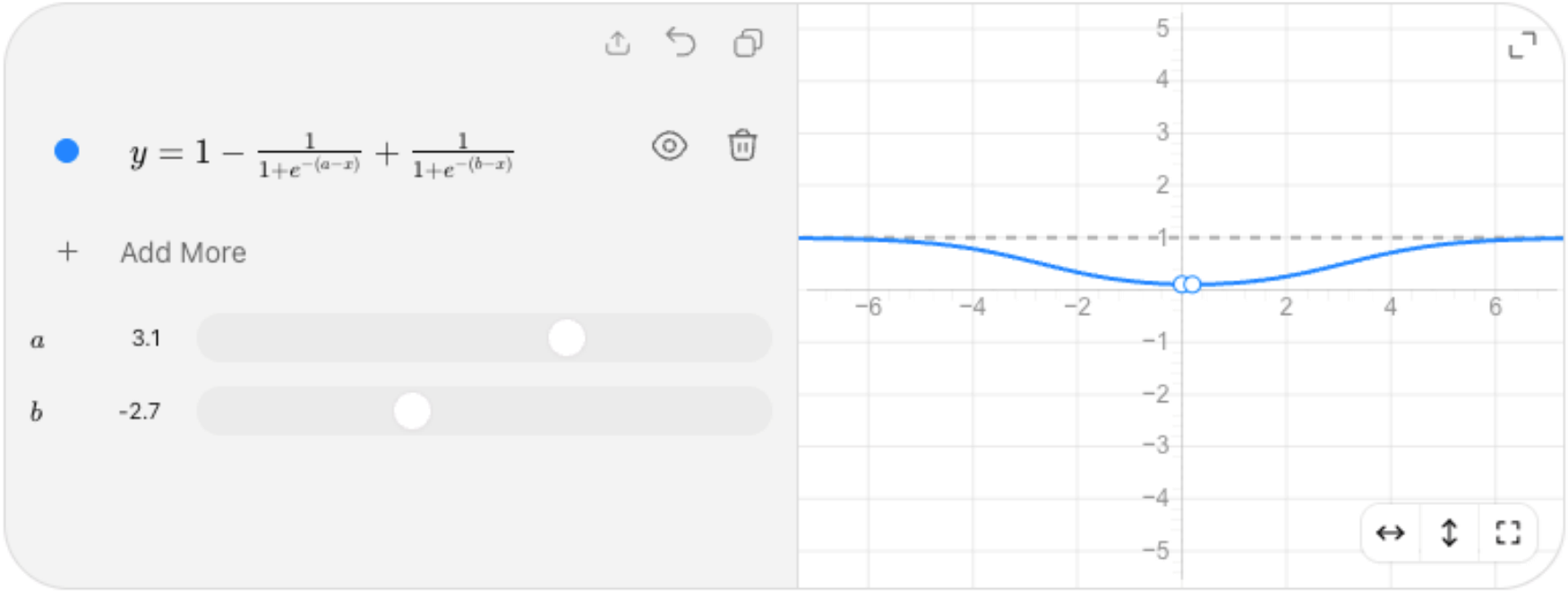
7

Learning the Discriminator: Error Function is Convex

https://chatgpt.com/s/w_6ae2afb0155881 91b86efb2e1b015c3e

$$
\begin{aligned}
e(w_1) &= [1.0 - f^+(\boldsymbol{o}^+)] + [f^+(\boldsymbol{o}^-) - 0.0] \\
&= [1.0 - f^+(a_1)] + [f^+(b_1)] \\
&= [1.0 - \sigma(a_1 - w_1)] + [\sigma(b_1 - w_1)]
\end{aligned}
$$





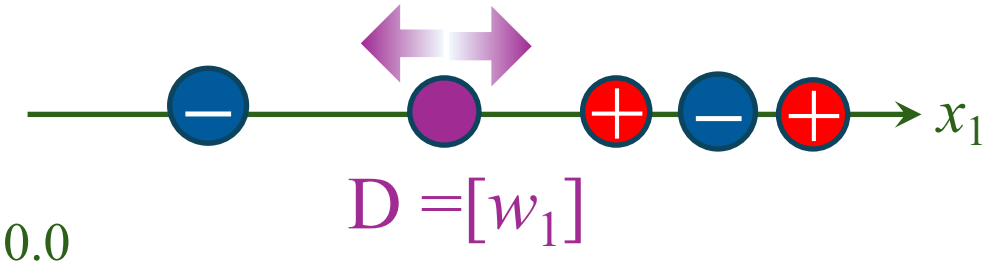


A **rope team** is a climbing technique where two or more climbers who are attached to a single climbing rope move simultaneously together along easy-angled terrain that does not require points of fixed climbing protection to be inserted along the route.

Learning the Discriminator: Multiple Observations

$$e(w_1) = \sum_{o_1 \in \mathbf{O}^+} [1.0 - f^+(\boldsymbol{o}^+)] + \sum_{o_1 \in \mathbf{O}^-} [f^+(\boldsymbol{o}^-) - 0.0] = 0.0$$

$$= \sum_{o_1 \in \mathbf{O}^+} [1.0 - \sigma(o_1 - w_1)] + \sum_{o_1 \in \mathbf{O}^-} [\sigma(o_1 - w_1) - 0.0] = 0.0$$



One **average** step down of pairs in subsets, aka **batches**:

$$w^{(1)}{}_1 \rightarrow w^{(0)}{}_1 - \eta \times \frac{1}{batch\ size} \sum_{a_1, b_1 \in batch\ 1} \frac{d\ e(w^{(0)}{}_1 : (a_1, b_1))}{d\ w_1}$$

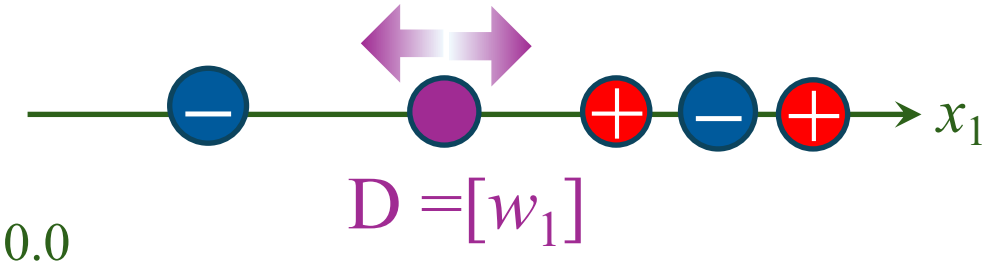
$$w^{(2)}{}_1 \rightarrow w^{(1)}{}_1 - \eta \times \frac{1}{batch\ size} \sum_{a_1, b_1 \in batch\ 2} \frac{d\ e(w^{(1)}{}_1 : (a_1, b_1))}{d\ w_1}$$

$$w^{(3)}{}_1 \rightarrow w^{(2)}{}_1 - \eta \times \frac{1}{batch\ size} \sum_{a_1, b_1 \in batch\ 3} \frac{d\ e(w^{(2)}{}_1 : (a_1, b_1))}{d\ w_1}$$

Learning the Discriminator: Multiple Rounds: Epochs

$$e(w_1) = \sum_{o_1 \in \mathbf{O}^+} [1.0 - f^+(\boldsymbol{o}^+)] + \sum_{o_1 \in \mathbf{O}^-} [f^+(\boldsymbol{o}^-) - 0.0] = 0.0$$

$$= \sum_{o_1 \in \mathbf{O}^+} [1.0 - \sigma(o_1 - w_1)] + \sum_{o_1 \in \mathbf{O}^-} [\sigma(o_1 - w_1) - 0.0] = 0.0$$

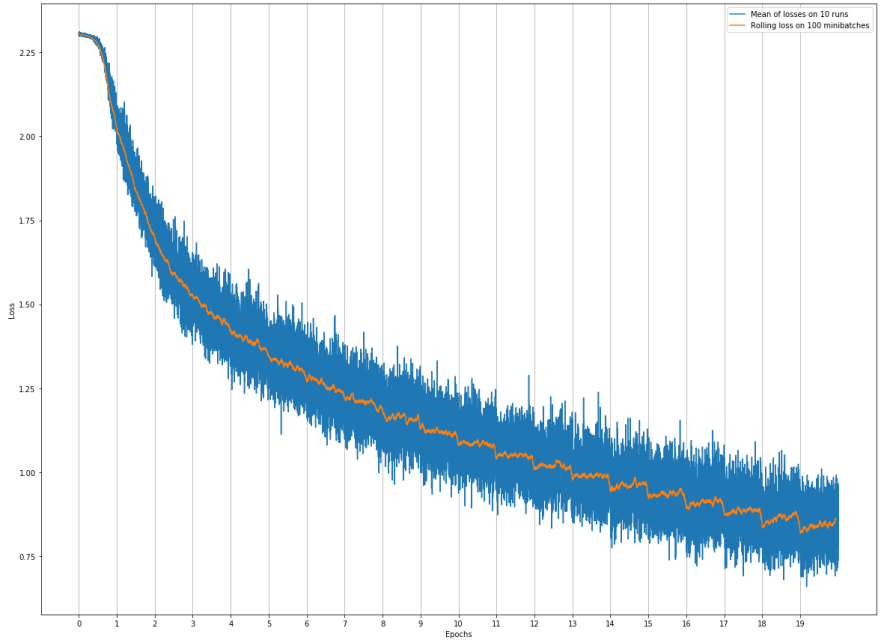


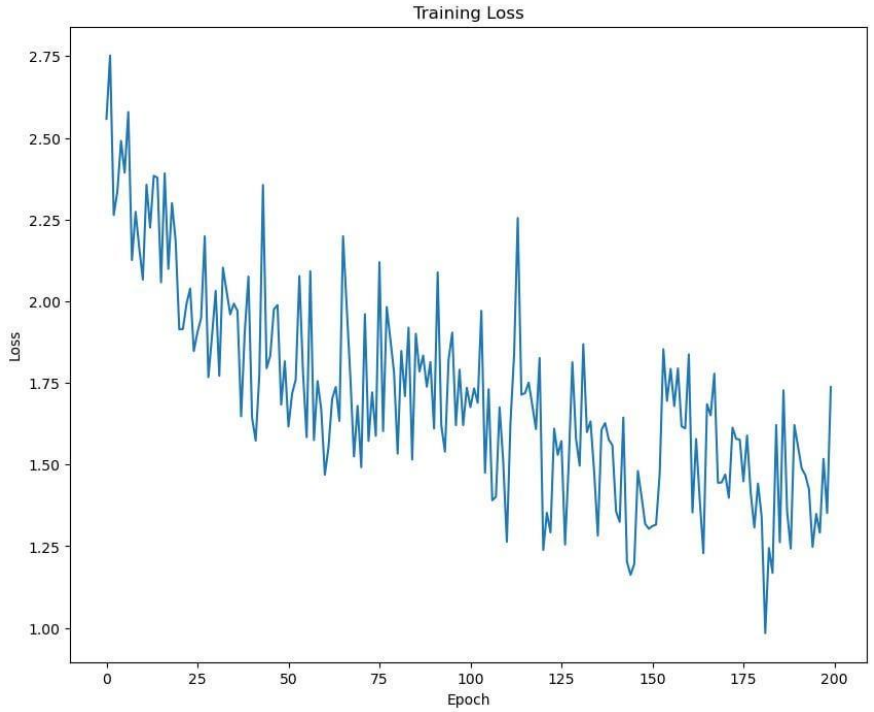
1st **full** consideration of all observations: **1 epoch**
2nd **full re-**consideration of all observations: **2 epochs**
3rd **full re-**consideration of all observations: **3 epochs**
...
N-th **full re-**consideration of all observations: **N epochs**

Learning the Discriminator: Training Error Trend

Error Function $e(w)$ = Loss Function $\mathcal{L}(w)$ = Objective Function

Step Down = Gradient Descend = A Technique to Optimize Functions





Learning the Discriminator: 2D Euclidian Space

A. Error $= [f^{+}(\boldsymbol{o}^{+}) - \text{'Positive'}] + [f^{+}(\boldsymbol{o}^{-}) - \text{'Negative'}]$

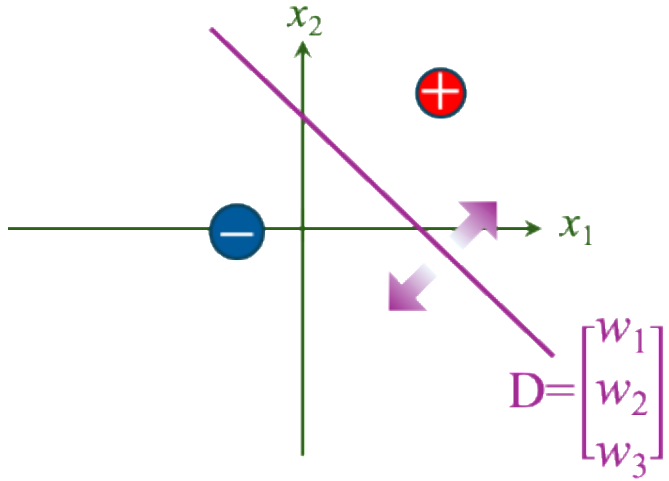
$$= [1.0 - f^{+}(\boldsymbol{o}^{+})] + [f^{+}(\boldsymbol{o}^{-}) - 0.0]$$

$$= [1.0 - \sigma([a_1 \quad a_2 \quad 1]\begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix})] + [\sigma([b_1 \quad b_2 \quad 1]\begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix})]$$

$$= e(w_1, w_2, w_3) \geq 0$$

B. Modify w_1, w_2, w_3 such that $e(w_1, w_2, w_3) \rightarrow 0.0$

C. Go back to A



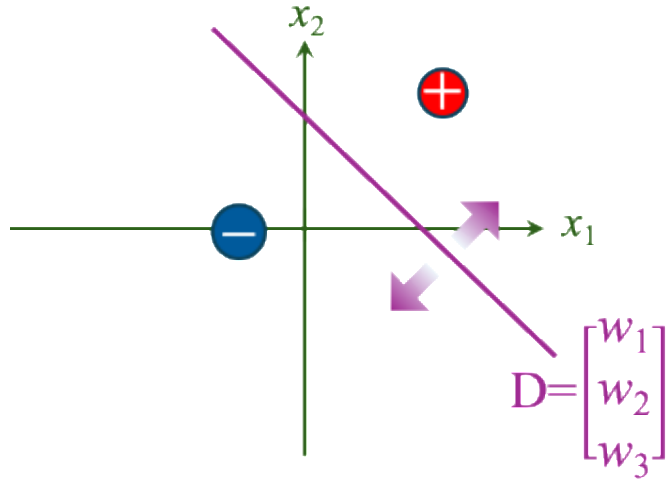
Mountain descending in 4D error space!

Learning the Discriminator: 2D Euclidian Space

$$e(w_1, w_2, w_3) = [1.0 - f^+(\boldsymbol{o}^+)] + [f^+(\boldsymbol{o}^-) - 0.0]$$

$$= [1.0 - \sigma([a_1 \quad a_2 \quad 1]\begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix})] + [\sigma([b_1 \quad b_2 \quad 1]\begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix})]$$

$$= 1.0 - \underbrace{\sigma(a_1 w_1 + a_2 w_2 + w_3)}_{\text{Forward pass for } \boldsymbol{o}^+} + \underbrace{\sigma(b_1 w_1 + b_2 w_2 + w_3)}_{\text{Forward pass for } \boldsymbol{o}^-}$$



14

Learning the Discriminator: 2D Euclidian Space

$$e(w_1, w_2, w_3) = [1.0 - f^+(\boldsymbol{o}^+)] + [f^+(\boldsymbol{o}^-) - 0.0]$$

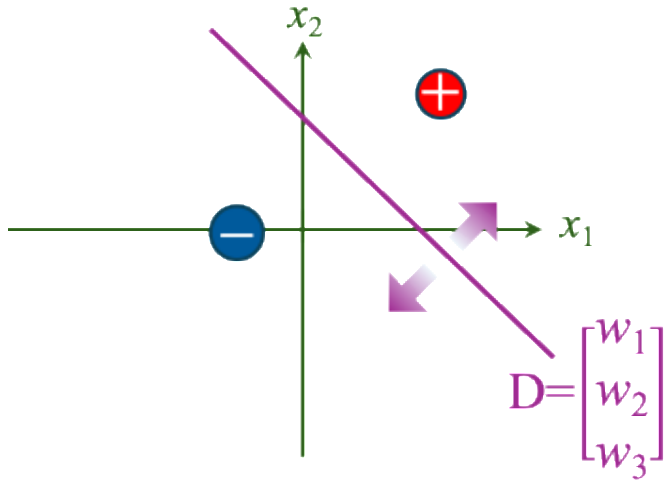
$$= [1.0 - \sigma([a_1 \quad a_2 \quad 1]\begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix})] + [\sigma([b_1 \quad b_2 \quad 1]\begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix})]$$

$$= 1.0 - \sigma(a_1 w_1 + a_2 w_2 + w_3) + \sigma(b_1 w_1 + b_2 w_2 + w_3)$$

$$w^{(i+1)}{}_1 \rightarrow w^{(i)}{}_1 - \eta \times \frac{d\, e(w^{(i)}{}_1 : (\boldsymbol{o}^+, \boldsymbol{o}^-))}{d\, w_1}$$

$$w^{(i+1)}{}_2 \rightarrow w^{(i)}{}_2 - \eta \times \frac{d\, e(w^{(i)}{}_2 : (\boldsymbol{o}^+, \boldsymbol{o}^-))}{d\, w_2}$$

$$w^{(i+1)}{}_3 \rightarrow w^{(i)}{}_3 - \eta \times \frac{d\, e(w^{(i)}{}_3 : (\boldsymbol{o}^+, \boldsymbol{o}^-))}{d\, w_3}$$



15

Learning the Discriminator: 2D Euclidian Space

$$e(w_1, w_2, w_3) = [1.0 - f^+(\boldsymbol{o}^+)] + [f^+(\boldsymbol{o}^-) - 0.0]$$

$$= [1.0 - \sigma([a_1 \quad a_2 \quad 1]\begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix})] + [\sigma([b_1 \quad b_2 \quad 1]\begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix})]$$

$$= 1.0 - \sigma(a_1 w_1 + a_2 w_2 + w_3) + \sigma(b_1 w_1 + b_2 w_2 + w_3)$$

$$\frac{d\, e(w^{(i)}_1: (\boldsymbol{o}^+, \boldsymbol{o}^-))}{d\, w_1} = -a_1\, \sigma(a_1 w_1 + a_2 w_2 + w_3)(1 - \sigma(a_1 w_1 + a_2 w_2 + w_3))$$

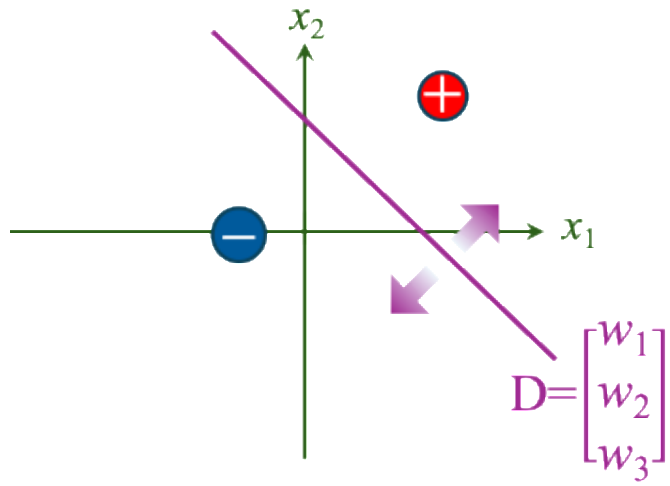
$$+ b_1\, \sigma(b_1 w_1 + b_2 w_2 + w_3)(1 - \sigma(b_1 w_1 + b_2 w_2 + w_3))$$

$$\frac{d\, e(w^{(i)}_2: (\boldsymbol{o}^+, \boldsymbol{o}^-))}{d\, w_2} = -a_2\, \sigma(a_1 w_1 + a_2 w_2 + w_3)(1 - \sigma(a_1 w_1 + a_2 w_2 + w_3))$$

$$+ b_2\, \sigma(b_1 w_1 + b_2 w_2 + w_3)(1 - \sigma(b_1 w_1 + b_2 w_2 + w_3))$$

$$\frac{d\, e(w^{(i)}_3: (\boldsymbol{o}^+, \boldsymbol{o}^-))}{d\, w_3} = -\sigma(a_1 w_1 + a_2 w_2 + w_3)(1 - \sigma(a_1 w_1 + a_2 w_2 + w_3))$$

$$+ \sigma(b_1 w_1 + b_2 w_2 + w_3)(1 - \sigma(b_1 w_1 + b_2 w_2 + w_3))$$



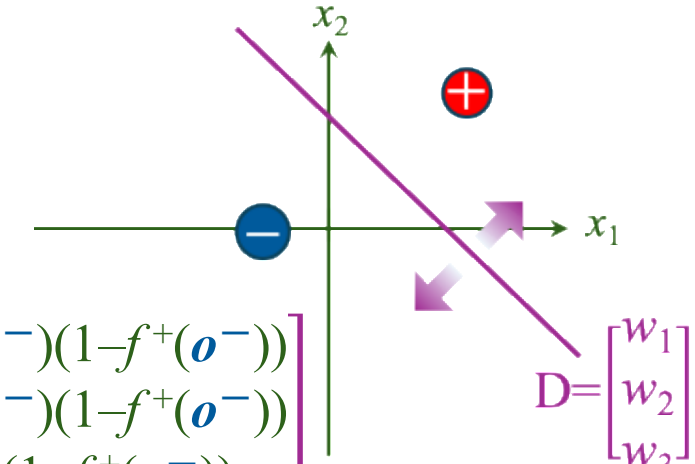
Learning the Discriminator: 2D Euclidian Space

$$e(w_1, w_2, w_3) = [1.0 - f^+(\boldsymbol{o}^+)] + [f^+(\boldsymbol{o}^-) - 0.0]$$

$$\frac{d\, e(w^{(i)}{}_1: (\boldsymbol{o}^+, \boldsymbol{o}^-))}{d\, w_1} = -\, a_1 f^+(\boldsymbol{o}^+)\,(1 - f^+(\boldsymbol{o}^+)) + b_1 f^+(\boldsymbol{o}^-)(1 - f^+(\boldsymbol{o}^-))$$

$$\frac{d\, e(w^{(i)}{}_2: (\boldsymbol{o}^+, \boldsymbol{o}^-))}{d\, w_2} = -\, a_2 f^+(\boldsymbol{o}^+)\,(1 - f^+(\boldsymbol{o}^+)) + b_2 f^+(\boldsymbol{o}^-)(1 - f^+(\boldsymbol{o}^-))$$

$$\frac{d\, e(w^{(i)}{}_3: (\boldsymbol{o}^+, \boldsymbol{o}^-))}{d\, w_3} = -\, f^+(\boldsymbol{o}^+)(1 - f^+(\boldsymbol{o}^+)) + f^+(\boldsymbol{o}^-)(1 - f^+(\boldsymbol{o}^-))$$



Gradient: differentiations across multiple dimensions

$$\nabla f(\boldsymbol{w}^{(i)}: (\boldsymbol{o}^+, \boldsymbol{o}^-)) = \frac{\partial\, e(\boldsymbol{w}^{(i)}: (\boldsymbol{o}^+, \boldsymbol{o}^-))}{\partial\, \boldsymbol{w}} = \begin{bmatrix} -\, a_1 f^+(\boldsymbol{o}^+)(1 - f^+(\boldsymbol{o}^+)) + b_1 f^+(\boldsymbol{o}^-)(1 - f^+(\boldsymbol{o}^-)) \\ -\, a_2 f^+(\boldsymbol{o}^+)(1 - f^+(\boldsymbol{o}^+)) + b_2 f^+(\boldsymbol{o}^-)(1 - f^+(\boldsymbol{o}^-)) \\ -\, f^+(\boldsymbol{o}^+)(1 - f^+(\boldsymbol{o}^+)) + f^+(\boldsymbol{o}^-)(1 - f^+(\boldsymbol{o}^-)) \end{bmatrix}$$

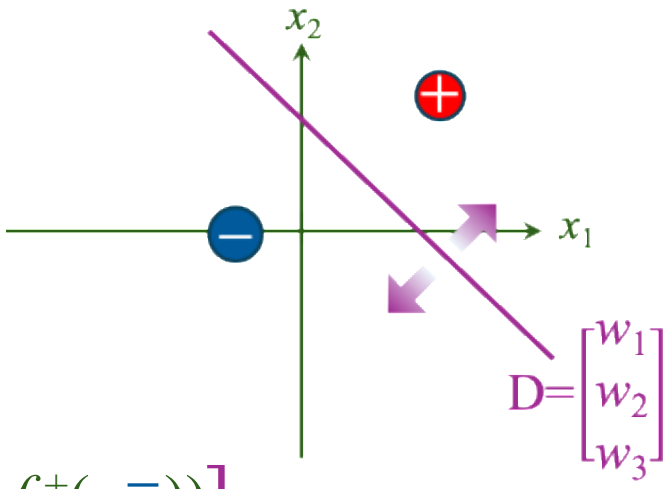
nabla (*aka del*)

17

Learning the Discriminator: 2D Euclidian Space

$$e(w_1, w_2, w_3) = [1.0 - f^+(\boldsymbol{o}^+)] + [f^+(\boldsymbol{o}^-) - 0.0]$$

$$\boldsymbol{w}^{(i+1)} \rightarrow \boldsymbol{w}^{(i)} - \eta \times \nabla e(\boldsymbol{w}^{(i)}: (\boldsymbol{o}^+, \boldsymbol{o}^-))$$

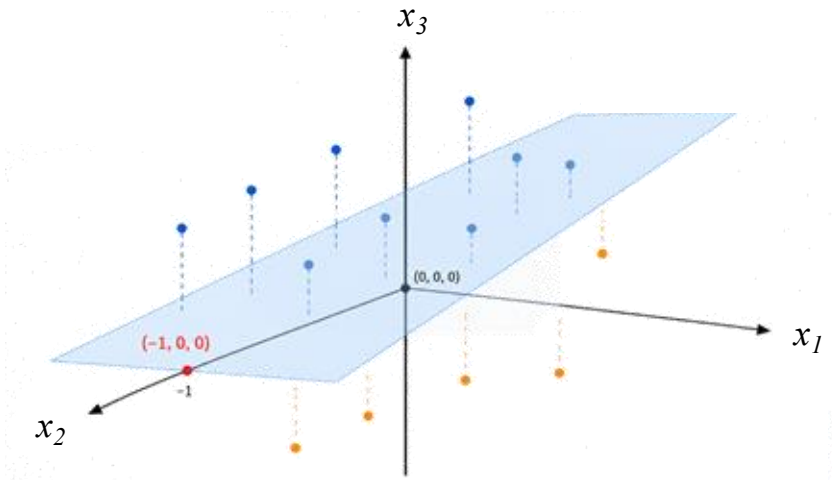


$$\begin{bmatrix} w^{(i+1)}{}_1 \\ w^{(i+1)}{}_2 \\ w^{(i+1)}{}_3 \end{bmatrix} \rightarrow \begin{bmatrix} w^{(i)}{}_1 \\ w^{(i)}{}_2 \\ w^{(i)}{}_3 \end{bmatrix} - \eta \times \begin{bmatrix} -a_1 f^+(\boldsymbol{o}^+)(1-f^+(\boldsymbol{o}^+)) + b_1 f^+(\boldsymbol{o}^-)(1-f^+(\boldsymbol{o}^-)) \\ -a_2 f^+(\boldsymbol{o}^+)(1-f^+(\boldsymbol{o}^+)) + b_2 f^+(\boldsymbol{o}^-)(1-f^+(\boldsymbol{o}^-)) \\ -f^+(\boldsymbol{o}^+)(1-f^+(\boldsymbol{o}^+)) + f^+(\boldsymbol{o}^-)(1-f^+(\boldsymbol{o}^-)) \end{bmatrix}$$

Learning the Discriminator: 3D Euclidian Space

$$e(w_1, w_2, w_3, w_4) = [1.0 - f^+(\boldsymbol{o}^+)] + [f^+(\boldsymbol{o}^-) - 0.0]$$

$$\boldsymbol{w}^{(i+1)} \rightarrow \boldsymbol{w}^{(i)} - \eta \times \nabla e(\boldsymbol{w}^{(i)}: (\boldsymbol{o}^+, \boldsymbol{o}^-))$$



$$\begin{bmatrix} w^{(i+1)}{}_1 \\ w^{(i+1)}{}_2 \\ w^{(i+1)}{}_3 \\ w^{(i+1)}{}_4 \end{bmatrix} \rightarrow \begin{bmatrix} w^{(i)}{}_1 \\ w^{(i)}{}_2 \\ w^{(i)}{}_3 \\ w^{(i)}{}_4 \end{bmatrix} - \eta \times \begin{bmatrix} -a_1 f^+(\boldsymbol{o}^+)(1-f^+(\boldsymbol{o}^+)) + b_1 f^+(\boldsymbol{o}^-)(1-f^+(\boldsymbol{o}^-)) \\ -a_2 f^+(\boldsymbol{o}^+)(1-f^+(\boldsymbol{o}^+)) + b_2 f^+(\boldsymbol{o}^-)(1-f^+(\boldsymbol{o}^-)) \\ -a_3 f^+(\boldsymbol{o}^+)(1-f^+(\boldsymbol{o}^+)) + b_3 f^+(\boldsymbol{o}^-)(1-f^+(\boldsymbol{o}^-)) \\ -f^+(\boldsymbol{o}^+)(1-f^+(\boldsymbol{o}^+)) + f^+(\boldsymbol{o}^-)(1-f^+(\boldsymbol{o}^-)) \end{bmatrix}$$

Mountain descending in 5D error space!

Learning the Discriminator: nD Euclidian Space

$$e(w_1, w_2, w_3, w_4, \ldots, w_{n+1}) = [1.0 - f^+(\boldsymbol{o}^+)] + [f^+(\boldsymbol{o}^-) - 0.0]$$

$$\boldsymbol{w}^{(i+1)} \rightarrow \boldsymbol{w}^{(i)} - \eta \times \nabla e(\boldsymbol{w}^{(i)}: (\boldsymbol{o}^+, \boldsymbol{o}^-))$$

$$\begin{bmatrix} w^{(i+1)}{}_1 \\ w^{(i+1)}{}_2 \\ w^{(i+1)}{}_3 \\ w^{(i+1)}{}_4 \\ \\ \cdots \\ \\ w^{(i+1)}{}_{n+1} \end{bmatrix} \rightarrow \begin{bmatrix} w^{(i)}{}_1 \\ w^{(i)}{}_2 \\ w^{(i)}{}_3 \\ w^{(i)}{}_4 \\ \\ \cdots \\ \\ w^{(i)}{}_{n+1} \end{bmatrix} - \eta \times \begin{bmatrix} -a_1 f^+(\boldsymbol{o}^+)(1-f^+(\boldsymbol{o}^+)) + b_1 f^+(\boldsymbol{o}^-)(1-f^+(\boldsymbol{o}^-)) \\ -a_2 f^+(\boldsymbol{o}^+)(1-f^+(\boldsymbol{o}^+)) + b_2 f^+(\boldsymbol{o}^-)(1-f^+(\boldsymbol{o}^-)) \\ -a_3 f^+(\boldsymbol{o}^+)(1-f^+(\boldsymbol{o}^+)) + b_3 f^+(\boldsymbol{o}^-)(1-f^+(\boldsymbol{o}^-)) \\ -a_4 f^+(\boldsymbol{o}^+)(1-f^+(\boldsymbol{o}^+)) + b_4 f^+(\boldsymbol{o}^-)(1-f^+(\boldsymbol{o}^-)) \\ \cdots \\ -f^+(\boldsymbol{o}^+)(1-f^+(\boldsymbol{o}^+)) + f^+(\boldsymbol{o}^-)(1-f^+(\boldsymbol{o}^-)) \end{bmatrix}$$

Mountain descending in $(n+2)$D error space!

Outline: W08C

I) Task: Classification

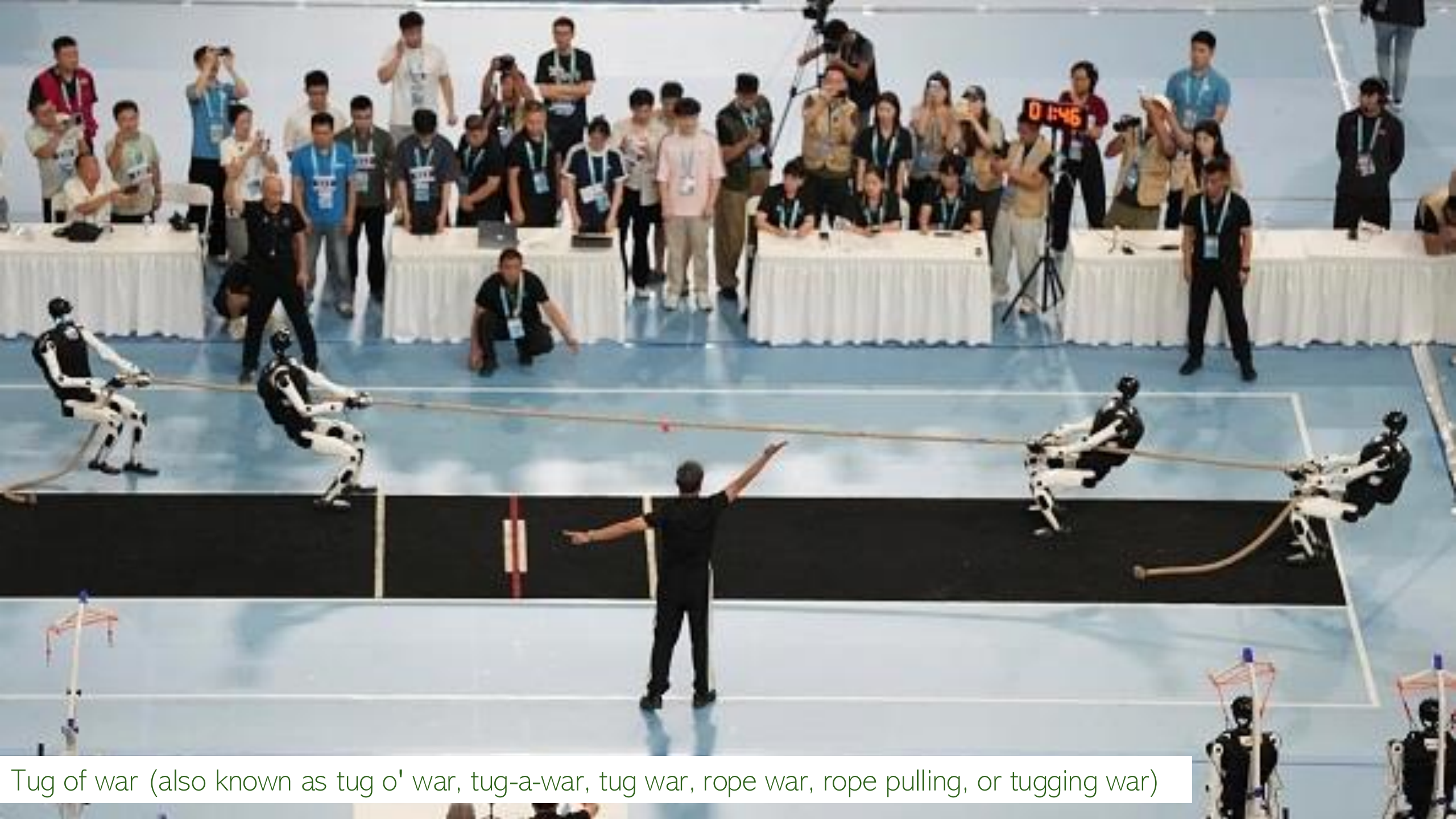
II) Supervision: Fully Supervised

III) Technique: Logistic Regression

- Learning the Discriminator from Training Data (from Experience)
- Mountain Descending
- Hyperparameters

IIII) Evaluation

- Metrics (Few New Metrics), Methodology



Tug of war (also known as tug o' war, tug-a-war, tug war, rope war, rope pulling, or tugging war)

Logistic Regression: Let's zoom in …

$$f_{\boldsymbol{\Theta}}^{+}(\mathbf{o}) = \sigma([o_1 \;\; o_2 \; \ldots \; o_n \; 1] \begin{bmatrix} w_1 \\ w_2 \\ \\ \ldots \\ \\ w_{n+1} \end{bmatrix}) = \text{Probability of being positive}$$



Parametric Model: Training Time

We have parameters $\boldsymbol{\Theta}=\{w_1, w_2, w_3, w_4, \ldots, w_{n+1}\}$ to learn: $f_{\boldsymbol{\Theta}}^{+}(\mathbf{o})$

Logistic Regression: Let's zoom in …

$$f_{\Theta}^{+}(\mathbf{o}) = \sigma([o_1 \;\; o_2 \; \ldots \; o_n \; 1] \begin{bmatrix} w_1 \\ w_2 \\ \\ \ldots \\ \\ w_{n+1} \end{bmatrix}) \begin{cases} \geq 0.50\text{: Positive} \\ < 0.50\text{: Negative} \end{cases}$$



Constant Inference Time

"multiply the n-dimensional point of the unseen observation with the model (discriminator)'s parameters and then squash it by logistic regression:

- *If above 0.5 probability (likelihood) of being positive → positive*
- *If below 0.5 probability (likelihood) of being positive → negative"*

Logistic Regression: Let's zoom in …

$$f_{\Theta}^{+}(\mathbf{o}) = \sigma([o_1 \;\; o_2 \; \ldots \; o_n \; 1] \begin{bmatrix} w_1 \\ w_2 \\ \\ \ldots \\ \\ w_{n+1} \end{bmatrix}) \begin{cases} \geq 0.50\text{: Positive} \\ < 0.50\text{: Negative} \end{cases}$$

Any hyperparameters to tune? *Not for the model itself! But …*

Logistic Regression: Let's zoom in …

$$f_{\Theta}^{+}(\mathbf{o}) = \sigma([o_1 \;\; o_2 \; \ldots \; o_n \; 1] \begin{bmatrix} w_1 \\ w_2 \\ \\ \ldots \\ \\ w_{n+1} \end{bmatrix}) \begin{cases} \geq 0.50\text{: Positive} \\ < 0.50\text{: Negative} \end{cases}$$

Any hyperparameters to tune?

→ Inference thresholding: Large (0.90) vs. (0.10)

Logistic Regression: Let's zoom in …

$$f_{\Theta}^{+}(\mathbf{o}) = \sigma([o_1 \;\; o_2 \; \ldots \; o_n \; 1] \begin{bmatrix} w_1 \\ w_2 \\ \\ \ldots \\ \\ w_{n+1} \end{bmatrix}) \begin{cases} \geq 0.50\text{: Positive} \\ < 0.50\text{: Negative} \end{cases}$$

Any hyperparameters to tune?

→ Training (learning from training set) has hyperparameters …

→ Learning rate (η): High (1.0) vs. Low (0.001)

→ Batch size: Large (entire training set) vs. Small (1 observation)

→ Number of epochs: Many (100) vs. Few (1)

27

Logistic Regression: Let's zoom in …

Cross-Fold-Validation-LR.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[18]

```python
# Train logistic regression using mini-batch gradient descent
def train_lr(X, y_train, learning_rate, epochs, batch_size):
    n = len(X)
    w = np.random.uniform(-0.01, 0.01, size=3)# Randomly initialize (w1, w2, w3)
    rng = np.random.default_rng(2026)

    for epoch in range(epochs):
        order = rng.permutation(n)

        for start in range(0, n, batch_size):
            idx = order[start:start + batch_size]

            X_batch = X[idx]
            y_batch = y_train[idx]

            X_augmented = np.column_stack((X_batch, np.ones(len(X_batch))))# Add constant 1: [x1, x2, 1], homogenious point
            y_pred = sigmoid(X_augmented @ w)# forward pass

            # Hossein's Proposed Error (loss) Function: E = (1 - y_pred) for positive + y_pred for negative

            # if y_batch = 1 -> (1-2*1) = -1 but if y_batch = 0 -> (1-2*0) = 1
            g = (1 - 2 * y_batch) * y_pred * (1 - y_pred) # Positive: -y_pred * (1-y_pred), Negative: +y_pred * (1-y_pred)

            dw = X_augmented.T @ g / len(y_batch) # Gradient for w1, w2, and w3
            w -= learning_rate * dw # Gradient descent

    return w
```

Outline: W08C

I) Task: Classification

II) Supervision: Fully Supervised

III) Technique: Logistic Regression

- Learning the Discriminator from Training Data (from Experience)
- Mountain Descending
- Hyperparameters
- Common Error (Loss) Functions, Multiclass, non-Linear Discriminators

IIII) Evaluation

- Metrics (Few New Metrics), Methodology